Compilation Process in C Program ⚙️

From Source Code to Executable

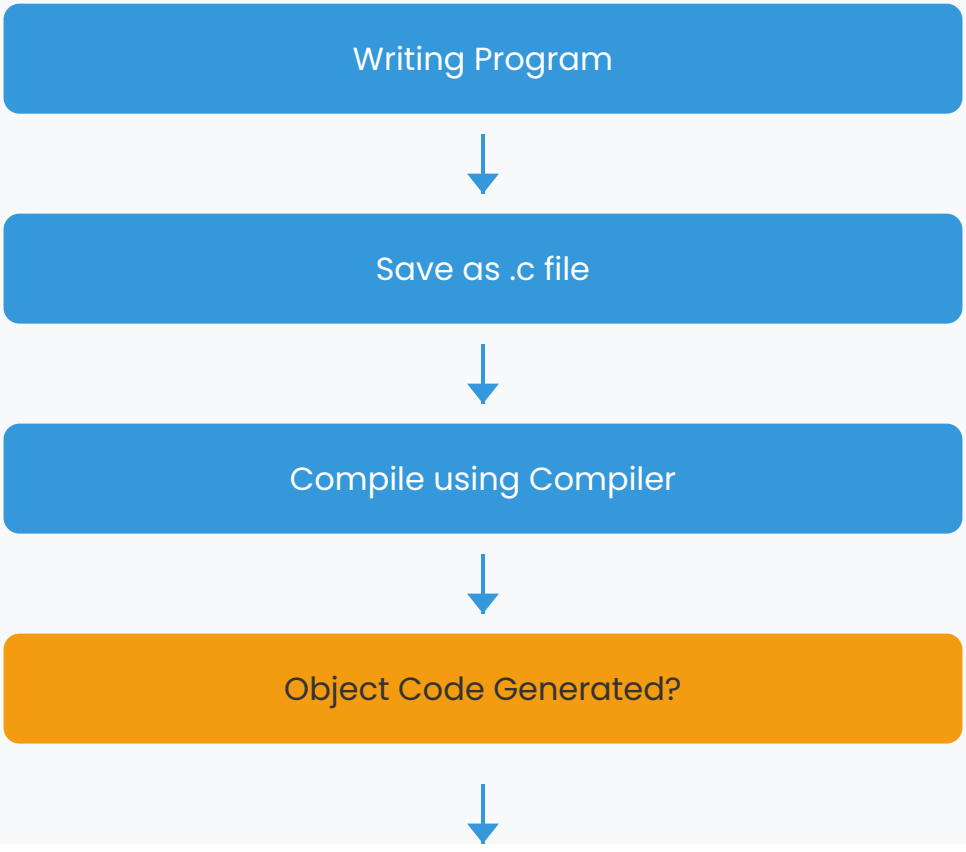
Learn how C programs are transformed from human-readable code to machine-executable files!

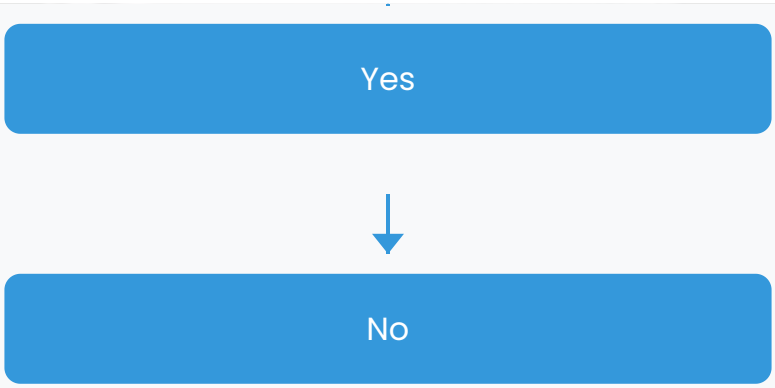
Compilation Process Overview 🔄

After you've finished writing the program, save it in a file with the extension .c. This software is written in a high-level language. However, the computer does not understand this language.

The following step is to translate the high-level language program (source code) to machine language (object code). A compiler is a piece of software or a program that handles this duty.

Every programming language has its own compiler, which turns source code to object code. If the program is syntactically accurate, the compiler will successfully compile it; otherwise, the object code will not be created.





The Compiler 🛠️

UNIX Platform

If you are working on UNIX platform, then if the name of the program file is testprog.c, to compile it, the simplest method is to type:

Command

```
cc testprog.c
```

This will compile testprog.c, and, if successful, will produce a executable file called a.out.

If you want to give the executable file any other name, you can type:

Command

```
cc testprog.c -o testprog
```

This will compile testprog.c, creating an executable file testprog.

TurboC on DOS Platform

If you are working with TurboC on DOS platform then the option for compilation is provided on the menu.

If the program is syntactically correct then this will produce a file named as testprog.obj. If not, then the syntax errors will be displayed on the screen and the object file will not be produced.

The errors need to be removed before compiling the program again. This process of removing the errors from the program is called as the debugging.

Semantic and Syntax Errors ❌

Every language has an associated grammar, and the program written in that language has to follow the rules of that grammar.

For example in English a sentence such as "Raghav, is playing, with a ball". This sentence is syntactically incorrect because commas should not come the way they are in the sentence.

Likewise, C also follows certain syntax rules. When a C program is compiled, the compiler will check that the program is syntactically

correct. If there are any syntax errors in the program, those will be displayed on the screen with the corresponding line numbers.

⚠️ Syntax Errors

These are errors that violate the grammatical rules of the C language. The compiler will detect these errors and prevent the program from compiling successfully.

🔍 Semantic Errors

These mistakes are shown as warnings. These mistakes happen when a statement has no meaning. Although the program compiles with these mistakes, it is usually recommended that they be corrected as well, as they may cause difficulties during operation.

Example of Syntax Errors 🔍

Example: Write a program to print a message on the screen.

```c
/* Program to print a message on the screen*/
#include <stdio.h
main( )
{
```

```
    printf("Hello, how are you\n")
}
```

Errors:

Let the name of the program be test.c. If we compile the above program as it is we will get the following errors:

- Error test.c 1: No file name ending
- Error test.c 5: Statement missing ;
- Error test.c 6: Compound statement missing }

Corrected Version:

```
#include <stdio.h>
main( )
{
    printf("Hello, how are you\n");
}
```


Semantic Errors and Summary 📝

Apart from syntax mistakes, semantic errors are another form of error that appears during compilation. These mistakes are shown as warnings.

These mistakes happen when a statement has no meaning. Although the program compiles with these mistakes, it is usually

recommended that they be corrected as well, as they may cause difficulties during operation.

💡 Example of Semantic Error

As an example, suppose you declared a variable but did not utilize it, and you receive the warning "code has no effect." These variables are taking up unnecessary memory space.

Summary

The compilation process transforms your C source code into machine-executable code. Understanding the compilation process and how to handle both syntax and semantic errors is crucial for successful C programming.

Remember: Syntax errors prevent compilation, while semantic errors generate warnings but still allow compilation. Both should be addressed for optimal program performance.

Best Practice

🎯 Always address both syntax errors and semantic warnings to create clean, efficient, and error-free C programs.